

Assignment No: 1

Title of the Assignment: Linear regression by using Deep Neural network: Implement Boston housing price. Prediction problem by linear regression using Deep Neural network. Use Boston House price prediction dataset.

Objective of the Assignment: To perform Linear regression by using Deep Neural network on Boston House Dataset.

Outcome of the Assignment: To find the best scenario for the result to be achieved for a given data set using linear regression.

Prerequisite:

1. Basic of programming language
2. Concept of Linear Regression
3. Concept of Deep Neural Network

Theory: Linear regression is a statistical approach that is commonly used to model the relationship between a dependent variable and one or more independent variables. It assumes a linear relationship between the variables and uses mathematical methods to estimate the coefficients that best fit the data.

Deep neural networks are a type of machine learning algorithm that are modeled after the structure and function of the human brain. They consist of multiple layers of interconnected neurons that process data and learn from it to make predictions or classifications. Linear regression using deep neural networks combines the principles of linear regression with the power of deep learning algorithms. In this approach, the input features are passed through one or more layers of neurons to extract features and then a linear regression model is applied to the output of the last layer to make predictions. The weights and biases of the neural network are adjusted during training to optimize the performance of the model. This approach can be used for a variety of tasks, including predicting numerical values, such as stock prices or housing prices, and classifying data into categories, such as detecting whether an image contains a particular object or not. It is often used in fields such as finance, healthcare, and image recognition.

Example of Linear Regression:

A suitable example of linear regression using deep neural network would be predicting the price of a

house based on various features such as the size of the house, the number of bedrooms, the location, and the age of the house. In this example, the input features would be fed into a deep neural network, consisting of multiple layers of interconnected neurons. The first few layers of the network would learn to extract features from the input data, such as identifying patterns and correlations between the input features.

The output of the last layer would then be passed through a linear regression model, which would use the learned features to predict the price of the house. During training, the weights and biases of the neural network would be adjusted to minimize the difference between the predicted price and the actual price of the house. This process is known as gradient descent, and it involves iteratively adjusting the model's parameters until the optimal values are reached. Once the model is trained, it can be used to predict the price of a new house based on its features. This approach can be used in the real estate industry to provide accurate and reliable estimates of house prices, which can help both buyers and sellers make informed decisions.

Concept of Deep Neural Network- The classification of the Deep Neural Network is illustrated in Figure 1.

Refer Figure 1 for the detailed classification.

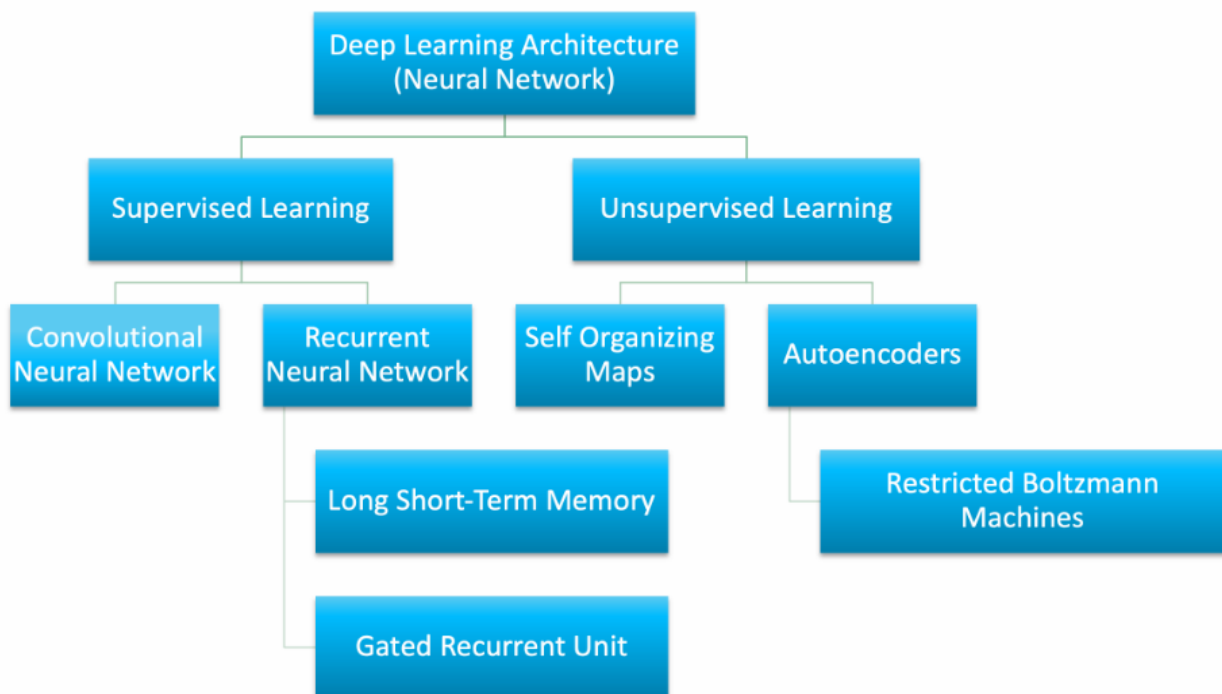


Figure1:DNN-CLASSIFICATION

A deep neural network is a type of machine learning algorithm that is modeled after the structure and function of the human brain. It consists of multiple layers of interconnected nodes, or artificial neurons, that process data and learn from it to make predictions or classifications. As shown in figure.

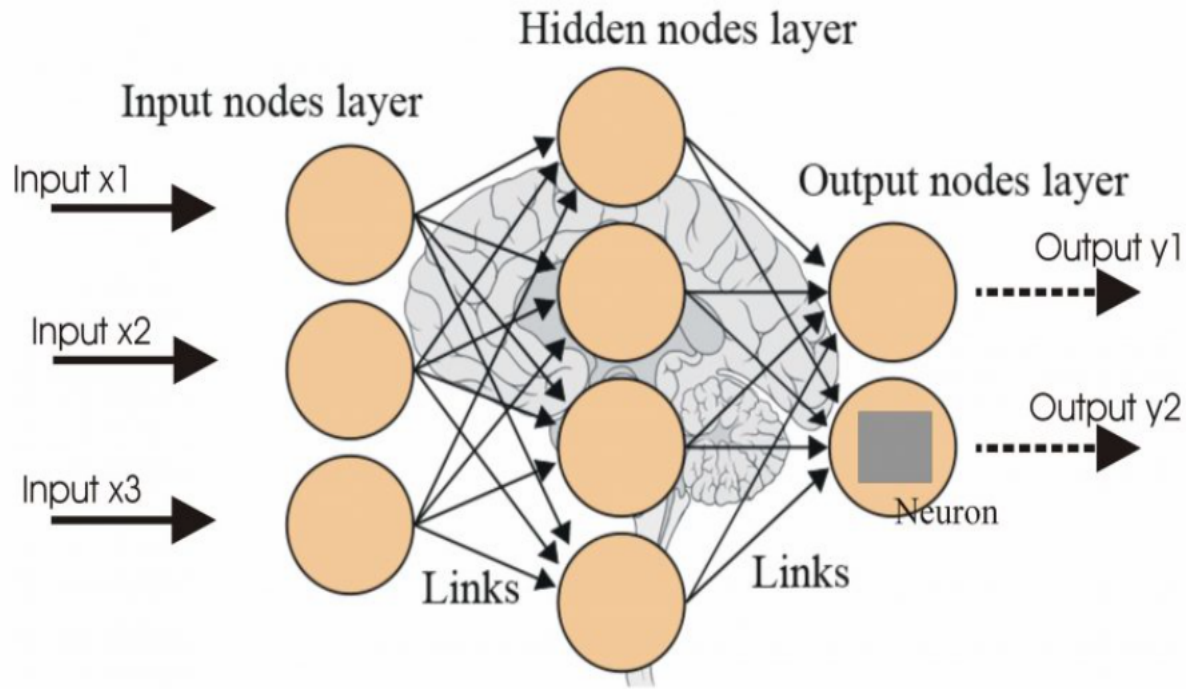


Figure 2: Deep Learning Architecture

Each layer of the network performs a specific type of processing on the data, such as identifying patterns or correlations between features, and passes the results to the next layer. The layers closest to the input are known as the "input layer", while the layers closest to the output are known as the "output layer".

The intermediate layers between the input and output layers are known as "hidden layers". These layers are responsible for extracting increasingly complex features from the input data, and can be deep (i.e., containing many hidden layers) or shallow (i.e., containing only a few hidden layers).

Deep neural networks are trained using a process known as backpropagation, which involves adjusting the weights and biases of the nodes based on the error between the predicted output and the actual output. This process is repeated for multiple iterations until the model reaches an optimal level of accuracy.

Deep neural networks are used in a variety of applications, such as image and speech recognition, natural language processing, and recommendation systems. They are capable of learning from vast amounts of data and can automatically extract features from raw data, making them a powerful tool for solving complex problems in a wide range of domains.

Deep Neural Network Working-

Boston House Price Prediction is a common example used to illustrate how a deep neural network can work for regression tasks. The goal of this task is to predict the price of a house in Boston based on various features such as the number of rooms, crime rate, and accessibility to public transportation.

Here's how a deep neural network can work for Boston House Price Prediction:

1. **Data preprocessing:** The first step is to preprocess the data. This involves normalizing the input features to have a mean of 0 and a standard deviation of 1, which helps the network learn more efficiently. The dataset is then split into training and testing sets.
2. **Model architecture:** A deep neural network is then defined with multiple layers. The first layer is the input layer, which takes in the normalized features. This is followed by several hidden layers, which can be deep or shallow. The last layer is the output layer, which predicts the house price.
3. **Model training:** The model is then trained using the training set. During training, the weights and biases of the nodes are adjusted based on the error between the predicted output and the actual output. This is done using an optimization algorithm such as stochastic gradient descent.
4. **Model evaluation:** Once the model is trained, it is evaluated using the testing set. The performance of the model is measured using metrics such as mean squared error or mean absolute error.
5. **Model prediction:** Finally, the trained model can be used to make predictions on new data, such as predicting the price of a new house in Boston based on its features.
6. By using a deep neural network for Boston House Price Prediction, we can obtain accurate predictions based on a large set of input features. This approach is scalable and can be used for other regression tasks as well.

Dataset Summary:

Source Code:

Output:

Conclusion: - In this manner, we can predict the Boston House Price using a Deep Neural Network. The linear regression model is applied to the given dataset, and the best-fit equation is calculated.

Assignment No: 2

Title of the Assignment: Binary classification using Deep Neural Networks Example: Classify movie reviews into positive" reviews and "negative" reviews, just based on the text content of the reviews. Use IMDB dataset

Objective of the Assignment: Students should be able to Classify movie reviews into positive reviews and "negative reviews on IMDB Dataset.

Prerequisite:

- Basic of programming language
- Concept of Classification
- Concept of Deep Neural Network

Theory:

Classification is a type of supervised learning in machine learning that involves categorizing data into predefined classes or categories based on a set of features or characteristics. It is used to predict the class of new, unseen data based on the patterns learned from the labeled training data.

In classification, a model is trained on a labeled dataset, where each data point has a known class label. The model learns to associate the input features with the corresponding class labels and can then be used to classify new, unseen data.

For example, we can use classification to identify whether an email is spam or not based on its content and metadata, to predict whether a patient has a disease based on their medical records and symptoms, or to classify images into different categories based on their visual features.

Classification algorithms can vary in complexity, ranging from simple models such as decision trees and k-nearest neighbors to more complex models such as support vector machines and neural networks. The choice of algorithm depends on the nature of the data, the size of the dataset, and the desired level of accuracy and interpretability.

Classification is a common task in deep neural networks, where the goal is to predict the class of an input based on its features. Here's an example of how classification can be performed in a deep neural network using the popular MNIST dataset of handwritten digits.

The MNIST dataset contains 60,000 training images and 10,000 testing images of handwritten digits from 0 to 9. Each image is a grayscale 28x28 pixel image, and the task is to classify each image into one

of the 10 classes corresponding to the 10 digits.

We can use a convolutional neural network (CNN) to classify the MNIST dataset. A CNN is a type of deep neural network that is commonly used for image classification tasks.

Working: Deep neural networks are commonly used for classification tasks because they can automatically learn to extract relevant features from raw input data and map them to the correct output class. The basic architecture of a deep neural network for classification consists of three main parts: an input layer, one or more hidden layers, and an output layer. The input layer receives the raw input data, which is usually preprocessed to a fixed size and format. The hidden layers are composed of neurons that apply linear transformations and nonlinear activations to the input features to extract relevant patterns and representations. Finally, the output layer produces the predicted class labels, usually as a probability distribution over the possible classes. During training, the deep neural network learns to adjust its weights and biases in each layer to minimize the difference between the predicted output and the true labels. This is typically done by optimizing a loss function that measures the discrepancy between the predicted and true labels, using techniques such as gradient descent or stochastic gradient descent.

One of the key advantages of deep neural networks for classification is their ability to learn hierarchical representations of the input data. In a deep neural network with multiple hidden layers, each layer learns to capture more complex and abstract features than the previous layer, by building on the representations learned by the earlier layers. This hierarchical structure allows deep neural networks to learn highly discriminative features that can separate different classes of input data, even when the data is highly complex or noisy. Overall, the effectiveness of deep neural networks for classification depends on the choice of architecture, hyperparameters, and training procedure, as well as the quality and quantity of the training data. When trained properly, deep neural networks can achieve state-of-the-art performance on a wide range of classification tasks, from image recognition to natural language processing.

IMDB Dataset-The IMDB dataset is a large collection of movie reviews collected from the IMDB website, which is a popular source of user-generated movie ratings and reviews. The dataset consists of 50,000 movie reviews, split into 25,000 reviews for training and 25,000 reviews for testing.

Each review is represented as a sequence of words, where each word is represented by an integer index based on its frequency in the dataset. The labels for each review are binary, with 0 indicating a negative review and 1 indicating a positive review.

The IMDB dataset is commonly used as a benchmark for sentiment analysis and text classification tasks, where the goal is to classify the movie reviews as either positive or negative based on their text content.

The dataset is challenging because the reviews are often highly subjective and can contain complex language and nuances of meaning, making it difficult for traditional machine learning approaches to accurately classify them. Deep learning approaches, such as deep neural networks, have achieved state-of-the-art performance on the IMDB dataset by automatically learning to extract relevant features from the raw text data and map them to the correct output class. The IMDB dataset is widely used in research and education for natural language processing and machine learning, as it provides a rich source of labeled text data for training and testing deep learning models.

Source Code and Output-

```
# The IMDB sentiment classification dataset consists of 50,000 movie reviews from IMDB users that are
labeled as either positive (1) or negative (0).
# The reviews are preprocessed and each one is encoded as a sequence of word indexes in the form of
integers.
# The words within the reviews are indexed by their overall frequency within the dataset. For example,
the integer "2" encodes the second most frequent word in the data.
# The 50,000 reviews are split into 25,000 for training and 25,000 for testing.
# Text Process word by word at different timestamp ( You may use RNN LSTM GRU )
# convert input text to vector represent input text
# DOMAIN: Digital content and entertainment industry
# CONTEXT: The objective of this project is to build a text classification model that analyses the
customer's sentiments based on their reviews in the IMDB database. The model uses a complex deep
learning model to build an embedding layer followed by a classification algorithm to analyse the
sentiment of the customers.
# DATA DESCRIPTION: The Dataset of 50,000 movie reviews from IMDB, labelled by sentiment
(positive/negative).
# Reviews have been preprocessed, and each review is encoded as a sequence of word indexes
(integers).
# For convenience, the words are indexed by their frequency in the dataset, meaning the word that has
index 1 is the most frequent word.
# Use the first 20 words from each review to speed up training, using a max vocabulary size of 10,000.
# As a convention, "0" does not stand for a specific word, but instead is used to encode any unknown
word.
# PROJECT OBJECTIVE: Build a sequential NLP classifier which can use input text parameters to
determine the customer sentiments.
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
#loading imdb data with most frequent 10000 words
```



```

from keras.datasets import imdb
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000) # you may take top 10,000
word frequently used review of movies other are discarded
#consolidating data for EDA Exploratory data analysis (EDA) is used by data scientists to analyze and
investigate data sets and summarize their main characteristics
data = np.concatenate((X_train, X_test), axis=0) # axis 0 is first running vertically downwards across
rows (axis 0), axis 1 is second running horizontally across columns (axis 1),
label = np.concatenate((y_train, y_test), axis=0)
X_train.shape
(25000,)
X_test.shape
(25000,)
y_train.shape
(25000,)
y_test.shape
(25000,)
print("Review is ",X_train[0]) # series of no converted word to vocabulary associated with index
print("Review is ",y_train[0])
Review is [1, 194, 1153, 194, 8255, 78, 228, 5, 6, 1463, 4369, 5012, 134, 26, 4, 715, 8, 118, 1634, 14,
394, 20, 13, 119, 954, 189, 102, 5, 207, 110, 3103, 21, 14, 69, 188, 8, 30, 23, 7, 4, 249, 126, 93, 4, 114,
9, 2300, 1523, 5, 647, 4, 116, 9, 35, 8163, 4, 229, 9, 340, 1322, 4, 118, 9, 4, 130, 4901, 19, 4, 1002, 5,
89, 29, 952, 46, 37, 4, 455, 9, 45, 43, 38, 1543, 1905, 398, 4, 1649, 26, 6853, 5, 163, 11, 3215, 2, 4,
1153, 9, 194, 775, 7, 8255, 2, 349, 2637, 148, 605, 2, 8003, 15, 123, 125, 68, 2, 6853, 15, 349, 165,
4362, 98, 5, 4, 228, 9, 43, 2, 1157, 15, 299, 120, 5, 120, 174, 11, 220, 175, 136, 50, 9, 4373, 228, 8255,
5, 2, 656, 245, 2350, 5, 4, 9837, 131, 152, 491, 18, 2, 32, 7464, 1212, 14, 9, 6, 371, 78, 22, 625, 64,
1382, 9, 8, 168, 145, 23, 4, 1690, 15, 16, 4, 1355, 5, 28, 6, 52, 154, 462, 33, 89, 78, 285, 16, 145, 95]
Review is 0
vocab=imdb.get_word_index() # Retrieve the word index file mapping words to indices
print(vocab)
{'fawn': 34701, 'tsukino': 52006, 'nunnery': 52007, 'sonja': 16816, 'vani': 63951, 'woods': 1408, 'spiders':
16115,
y_train
array([1, 0, 0, ..., 0, 1, 0])
y_test
array([0, 1, 1, ..., 0, 0, 0])
# Function to perform relevant sequence adding on the data
# Now it is time to prepare our data. We will vectorize every review and fill it with zeros so that it

```

contains exactly 10000 numbers.

That means we fill every review that is shorter than 500 with zeros.

We do this because the biggest review is nearly that long and every input for our neural network needs to have the same size.

We also transform the targets into floats.

sequences is name of method the review less than 10000 we perform padding overthere

binary vectorization code:

VECTORIZE as one cannot feed integers into a NN

Encoding the integer sequences into a binary matrix - one hot encoder basically

From integers representing words, at various lengths - to a normalized one hot encoded tensor (matrix) of 10k columns

`def vectorize(sequences, dimension = 10000):` # We will vectorize every review and fill it with zeros so that it contains exactly 10,000 numbers.

 # Create an all-zero matrix of shape (len(sequences), dimension)

`results = np.zeros((len(sequences), dimension))`

 for i, sequence in `enumerate(sequences)`:

`results[i, sequence] = 1`

`return results`

Now we split our data into a training and a testing set.

The training set will contain reviews and the testing set

Set a VALIDATION set

`test_x = data[:10000]`

`test_y = label[:10000]`

`train_x = data[10000:]`

`train_y = label[10000:]`

`test_x.shape`

`(10000,)`

`test_y.shape`

`(10000,)`

`train_x.shape`

`(40000,)`

`train_y.shape`

`(40000,)`

`print("Categories:", np.unique(label))`

`print("Number of unique words:", len(np.unique(np.hstack(data))))`

The `hstack()` function is used to stack arrays in sequence horizontally (column wise).

Categories: [0 1]

Number of unique words: 9998

```
length = [len(i) for i in data]
```

```
print("Average Review length:", np.mean(length))
```

```
print("Standard Deviation:", round(np.std(length)))
```

The whole dataset contains 9998 unique words and the average review length is 234 words, with a standard deviation of 173 words.

Average Review length: 234.75892

Standard Deviation: 173

If you look at the data you will realize it has been already pre-processed.

All words have been mapped to integers and the integers represent the words sorted by their frequency.

This is very common in text analysis to represent a dataset like this.

So 4 represents the 4th most used word,

5 the 5th most used word and so on...

The integer 1 is reserved for the start marker,

the integer 2 for an unknown word and 0 for padding.

Let's look at a single training example:

```
print("Label:", label[0])
```

Label: 1

```
print("Label:", label[1])
```

Label: 0

```
print(data[0])
```

Retrieves a dict mapping words to their index in the IMDB dataset.

```
index = imdb.get_word_index() # word to index
```

Create inverted index from a dictionary with document ids as keys and a list of terms as values for each document

```
reverse_index = dict([(value, key) for (key, value) in index.items()]) # id to word
```

```
decoded = " ".join([reverse_index.get(i - 3, "#") for i in data[0]])
```

The indices are offset by 3 because 0, 1 and 2 are reserved indices for "padding", "start of sequence" and "unknown".

```
print(decoded)
```

this film was just brilliant casting location scenery story direction everyone's really suited the part they

played and you could just imagine being there robert # is an amazing actor and now the same being director # father came from the same scottish island as myself so i loved the fact there was a real connection with this film the witty remarks throughout the film

#Adding sequence to data

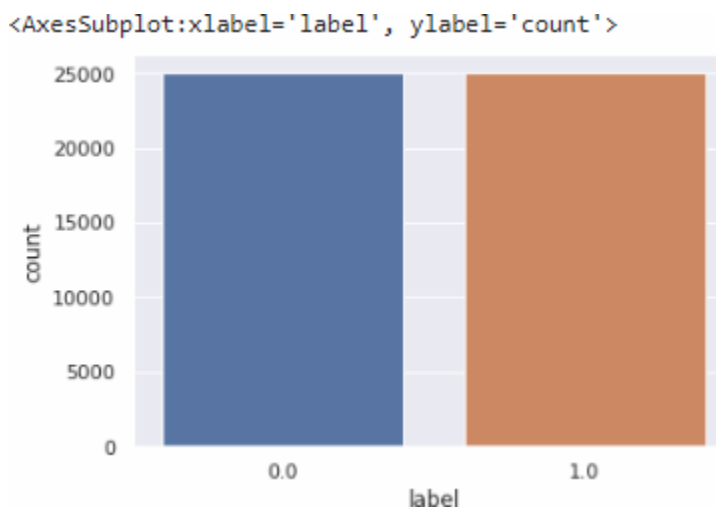
Vectorization is the process of converting textual data into numerical vectors and is a process that is usually applied once the text is cleaned.

```
data = vectorize(data)
```

```
label = np.array(label).astype("float32")
```

```
labelDF=pd.DataFrame({'label':label})
```

```
sns.countplot(x='label', data=labelDF)
```



Creating train and test data set

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(data,label, test_size=0.20, random_state=1)
```

```
X_train.shape
```

```
(40000, 10000)
```

```
X_test.shape
```

```
(10000, 10000)
```

Let's create sequential model

```
from keras.utils import to_categorical
```

```
from keras import models
```

```
from keras import layers
```

```
model = models.Sequential()
```

Input - Layer

Note that we set the input-shape to 10,000 at the input-layer because our reviews are 10,000 integers long.

The input-layer takes 10,000 as input and outputs it with a shape of 50.

```
model.add(layers.Dense(50, activation = "relu", input_shape=(10000, )))
```

Hidden - Layers

Please note you should always use a dropout rate between 20% and 50%. # here in our case 0.3 means 30% dropout we are using dropout to prevent overfitting.

By the way, if you want you can build a sentiment analysis without LSTMs, then you simply need to replace it by a flatten layer:

```
model.add(layers.Dropout(0.3, noise_shape=None, seed=None))
```

```
model.add(layers.Dense(50, activation = "relu"))
```

```
model.add(layers.Dropout(0.2, noise_shape=None, seed=None))
```

```
model.add(layers.Dense(50, activation = "relu"))
```

Output- Layer

```
model.add(layers.Dense(1, activation = "sigmoid"))
```

```
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 50)	500050
dropout (Dropout)	(None, 50)	0
dense_1 (Dense)	(None, 50)	2550
dropout_1 (Dropout)	(None, 50)	0
dense_2 (Dense)	(None, 50)	2550
dense_3 (Dense)	(None, 1)	51

```
Total params: 505,201
```

```
Trainable params: 505,201
```

```
Non-trainable params: 0
```

```
#For early stopping
```

```
# Stop training when a monitored metric has stopped improving.
```

```
# monitor: Quantity to be monitored.
```

```
# patience: Number of epochs with no improvement after which training will be stopped.
```

```
import tensorflow as tf
```

```
callback = tf.keras.callbacks.EarlyStopping(monitor='loss', patience=3)
```

```
# We use the "adam" optimizer, an algorithm that changes the weights and biases during training.
```

```
# We also choose binary-crossentropy as loss (because we deal with binary
```

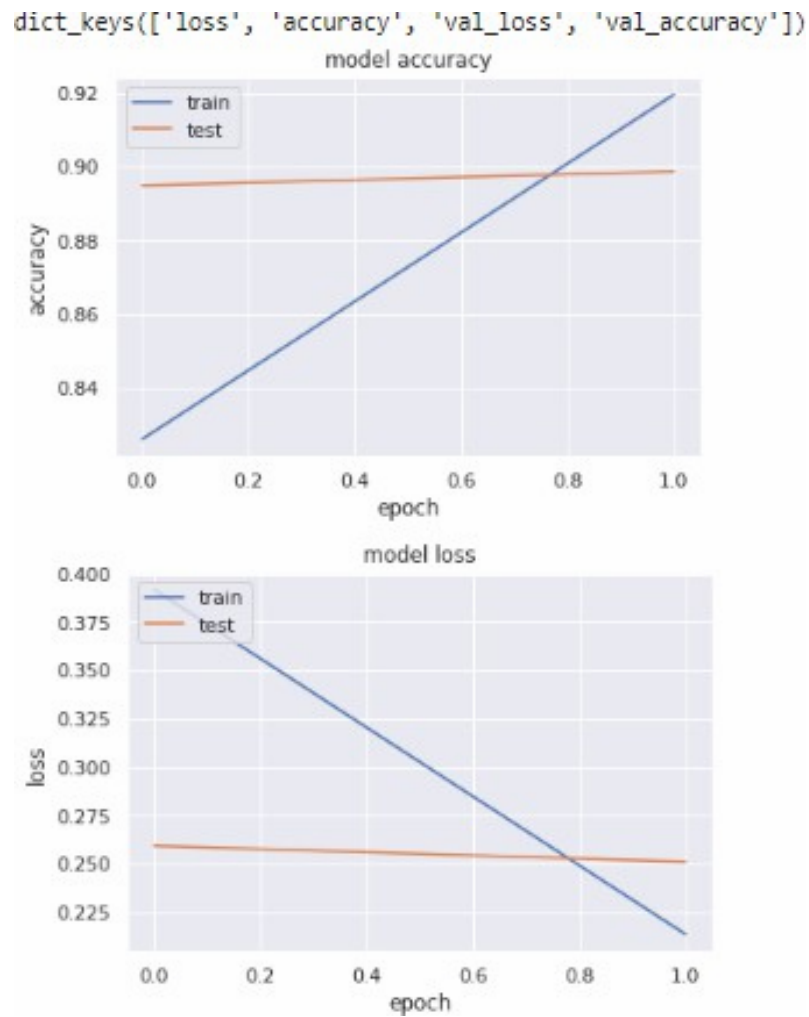

classification) and accuracy as our evaluation metric.

```
model.compile(
    optimizer = "adam",
    loss      = "binary_crossentropy",
    metrics   = ["accuracy"]
)
from sklearn.model_selection import train_test_split

results = model.fit(
    X_train,      y_train,
    epochs= 2,
    batch_size = 500,
    validation_data = (X_test, y_test),
    callbacks=[callback]
)
# Let's check mean accuracy of our model
print(np.mean(results.history["val_accuracy"]))
# Evaluate the model
score = model.evaluate(X_test, y_test, batch_size=500)
print('Test loss:', score[0])
print('Test accuracy:', score[1])
20/20 [=====] - 1s 24ms/step - loss: 0.2511 - accuracy:
0.8986
Test loss: 0.25108325481414795
Test accuracy: 0.8985999822616577
#Let's plot training history of our model.

# list all data in history
print(results.history.keys())
# summarize history for accuracy
plt.plot(results.history['accuracy'])
plt.plot(results.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
# summarize history for loss
plt.plot(results.history['loss'])
plt.plot(results.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
```

```
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```



Conclusion- In this way we can Classify the Movie Reviews by using DNN.

Assignment No: 3

Title of the Assignment: Use MNIST Fashion Dataset and create a classifier to classify fashion clothing into categories.

Objective of the Assignment: To classify movie reviews into positive reviews and "negative reviews on IMDB Dataset.

Prerequisite:

1. Basic of programming language
2. Concept of Classification
3. Concept of Deep Neural Network

Theory: Classification is a type of supervised learning in machine learning that involves categorizing data into predefined classes or categories based on a set of features or characteristics. It is used to predict the class of new, unseen data based on the patterns learned from the labeled training data. In classification, a model is trained on a labeled dataset, where each data point has a known class label. The model learns to associate the input features with the corresponding class labels and can then be used to classify new, unseen data. For example, we can use classification to identify whether an email is spam or not based on its content and metadata, to predict whether a patient has a disease based on their medical records and symptoms, or to classify images into different categories based on their visual features.

Classification algorithms can vary in complexity, ranging from simple models such as decision trees and k-nearest neighbors to more complex models such as support vector machines and neural networks. The choice of algorithm depends on the nature of the data, the size of the dataset, and the desired level of accuracy and interpretability.

Example- Classification is a common task in deep neural networks, where the goal is to predict the class of an input based on its features. Here's an example of how classification can be performed in a deep neural network using the popular MNIST dataset of handwritten digits.

The MNIST dataset contains 60,000 training images and 10,000 testing images of handwritten digits

from 0 to 9. Each image is a grayscale 28x28 pixel image, and the task is to classify each image into one of the 10 classes corresponding to the 10 digits. Convolutional neural network (CNN) is used to classify the MNIST dataset. A CNN is a type of deep neural network that is commonly used for image classification tasks.

CNN-

Convolutional Neural Networks (CNNs) are commonly used for image classification tasks, and they are designed to automatically learn and extract features from input images. Let's consider an example of using a CNN to classify images of handwritten digits. In a typical CNN architecture for image classification, there are several layers, including convolutional layers, pooling layers, and fully connected layers. Here's a diagram of a simple CNN architecture for the digit classification task:

The input to the network is an image of size 28x28 pixels, and the output is a probability distribution over the 10 possible digits (0 to 9). The convolutional layers in the CNN apply filters to the input image, looking for specific patterns and features. Each filter produces a feature map that highlights areas of the image that match the filter. The filters are learned during training, so the network can automatically learn which features are most relevant for the classification task. The pooling layers in the CNN downsample the feature maps, reducing the spatial dimensions of the data. This helps to reduce the number of parameters in the network, while also making the features more robust to small variations in the input image. The fully connected layers in the CNN take the flattened output from the last pooling layer and perform a classification task by outputting a probability distribution over the 10 possible digits.

During training, the network learns the optimal values of the filters and parameters by minimizing a loss function. This is typically done using stochastic gradient descent or a similar optimization algorithm.

Once trained, the network can be used to classify new images by passing them through the network and computing the output probability distribution. Overall, CNNs are powerful tools for image recognition tasks and have been used successfully in many applications, including object detection, face recognition, and medical image analysis.

CNNs have a wide range of applications in various fields, some of which are:

Image classification: CNNs are commonly used for image classification tasks, such as identifying objects in images and recognizing faces.

Object detection: CNNs can be used for object detection in images and videos, which involves identifying the location of objects in an image and drawing bounding boxes around them.

Semantic segmentation: CNNs can be used for semantic segmentation, which involves partitioning an image into segments and assigning each segment a semantic label (e.g., "road", "sky", "building").

Natural language processing: CNNs can be used for natural language processing tasks, such as

sentiment analysis and text classification.

Medical imaging: CNNs are used in medical imaging for tasks such as diagnosing diseases from X-rays and identifying tumors from MRI scans.

Autonomous vehicles: CNNs are used in autonomous vehicles for tasks such as object detection and lane detection.

Video analysis: CNNs can be used for tasks such as video classification, action recognition, and video captioning.

Overall, CNNs are a powerful tool for a wide range of applications, and they have been used successfully in many areas of research and industry.

Deep Neural Network Working on Classification using CNN-

Deep neural networks using CNNs work on classification tasks by learning to automatically extract features from input images and using those features to make predictions. Here's how it works:

Input layer: The input layer of the network takes in the image data as input.

Convolutional layers: The convolutional layers apply filters to the input images to extract relevant features. Each filter produces a feature map that highlights areas of the image that match the filter.

Activation functions: An activation function is applied to the output of each convolutional layer to introduce non-linearity into the network.

Pooling layers: The pooling layers downsample the feature maps to reduce the spatial dimensions of the data.

Dropout layer: Dropout is used to prevent overfitting by randomly dropping out a percentage of the neurons in the network during training.

Fully connected layers: The fully connected layers take the flattened output from the last pooling layer and perform a classification task by outputting a probability distribution over the possible classes.

Softmax activation function: The softmax activation function is applied to the output of the last fully connected layer to produce a probability distribution over the possible classes.

Loss function: A loss function is used to compute the difference between the predicted probabilities and the actual labels.

Optimization: An optimization algorithm, such as stochastic gradient descent, is used to minimize the loss function by adjusting the values of the network parameters.

Training: The network is trained on a large dataset of labeled images, adjusting the values of the parameters to minimize the loss function.

Prediction: Once trained, the network can be used to classify new images by passing them through the network and computing the output probability distribution.

MNIST Dataset-

The MNIST Fashion dataset is a collection of 70,000 grayscale images of 28x28 pixels, representing 10 different categories of clothing and accessories. The categories include T-shirts/tops, trousers, pullovers, dresses, coats, sandals, shirts, sneakers, bags, and ankle boots.

The dataset is often used as a benchmark for testing image classification algorithms, and it is considered a more challenging version of the original MNIST dataset which contains handwritten digits. The MNIST Fashion dataset was released by Zalando Research in 2017 and has since become a popular dataset in the machine learning community.

The MNIST Fashion dataset is a collection of 70,000 grayscale images of 28x28 pixels each. These images represent 10 different categories of clothing and accessories, with each category containing 7,000 images. The categories are as follows:

T-shirt/tops

Trousers

Pullovers

Dresses

Coats

Sandals

Shirts

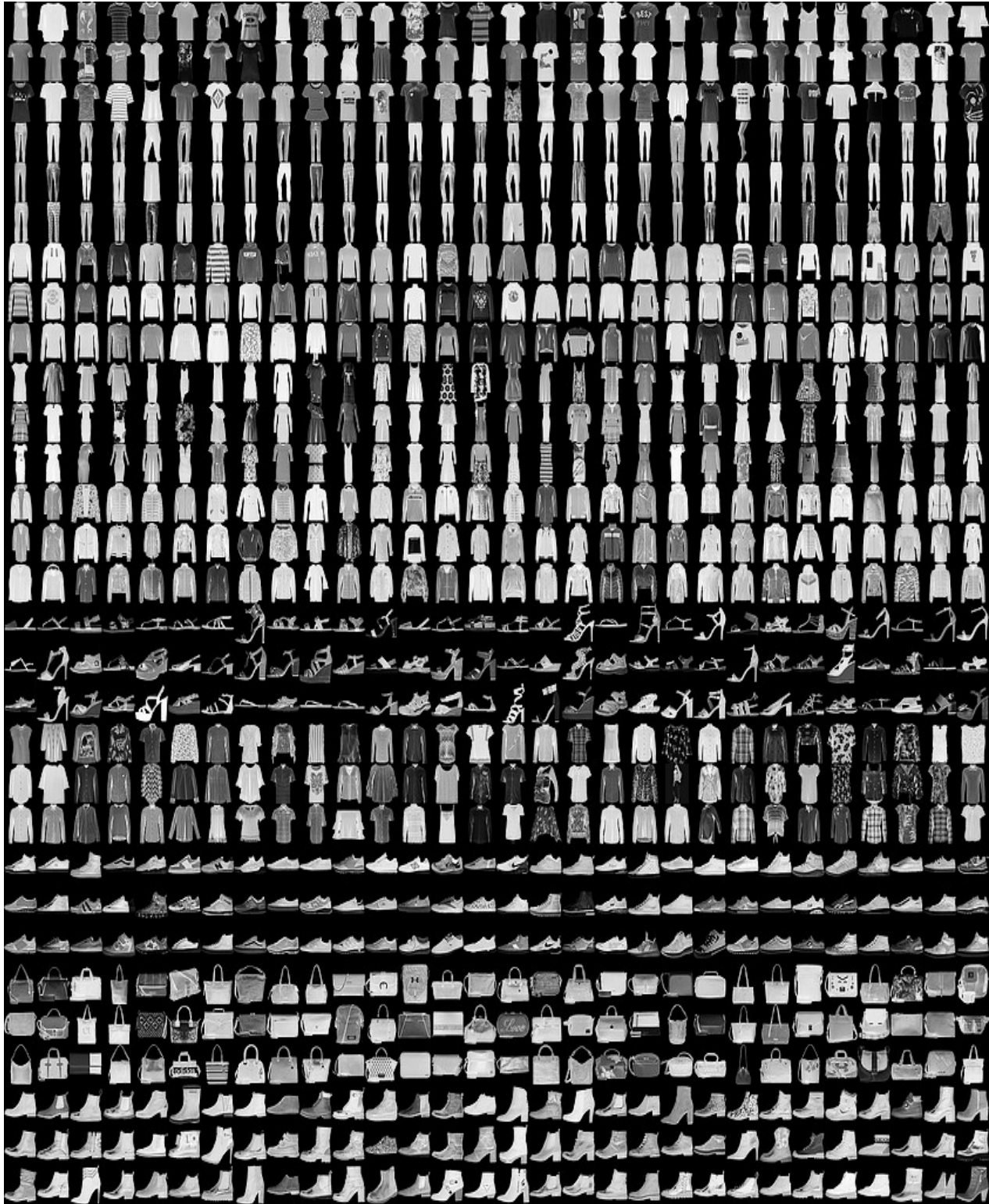
Sneakers

Bags

Ankle boots

The images were obtained from Zalando's online store and are preprocessed to be normalized and centered. The training set contains 60,000 images, while the test set contains 10,000 images. The goal of the dataset is to accurately classify the images into their respective categories.

The MNIST Fashion dataset is often used as a benchmark for testing image classification algorithms, and it is considered a more challenging version of the original MNIST dataset which contains handwritten digits. The dataset is widely used in the machine learning community for research and educational purposes.



General steps to perform Convolutional Neural Network (CNN) on the MNIST Fashion dataset:

- Import the necessary libraries, including TensorFlow, Keras, NumPy, and Matplotlib.
- Load the dataset using Keras' built-in function, `keras.datasets.fashion_mnist.load_data()`. This will provide the training and testing sets, which will be used to train and evaluate the CNN.
- Preprocess the data by normalizing the pixel values between 0 and 1, and reshaping the images to be of size (28, 28, 1) for compatibility with the CNN.
- Define the CNN architecture, including the number and size of filters, activation functions, and pooling layers. This can vary based on the specific problem being addressed.
- Compile the model by specifying the loss function, optimizer, and evaluation metrics. Common choices include categorical cross-entropy, Adam optimizer, and accuracy metric.
- Train the CNN on the training set using the `fit()` function, specifying the number of epochs and batch size.
- Evaluate the performance of the model on the testing set using the `evaluate()` function. This will provide metrics such as accuracy and loss on the test set.
- Use the trained model to make predictions on new images, if desired, using the `predict()` function.

Source Code with Output-

```
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
import numpy as np
```

```
(x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()
```

There are 10 image classes in this dataset and each class has a mapping corresponding to the following labels:

```
#0 T-shirt/top
#1 Trouser
#2 pullover
#3 Dress
#4 Coat
#5 sandals
```

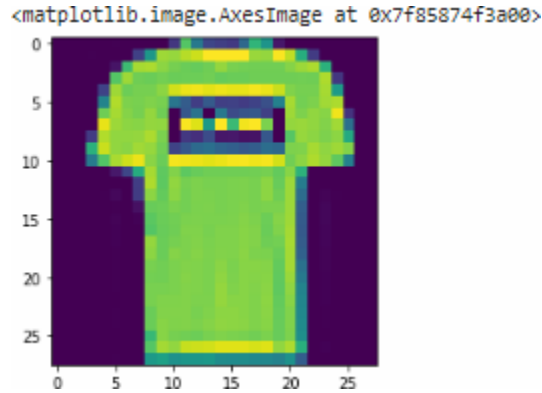
#6 shirt

#7 sneaker

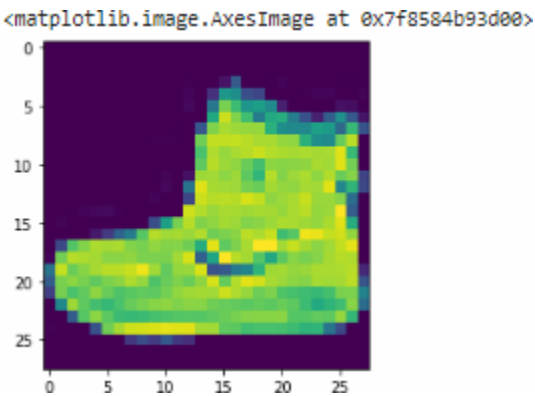
#8 bag

#9 ankle boot

```
plt.imshow(x_train[1])
```



```
plt.imshow(x_train[0])
```



Next, we will preprocess the data by scaling the pixel values to be between 0 and 1, and then reshaping the images to be 28x28 pixels.

```
x_train = x_train.astype('float32') / 255.0
```

```
x_test = x_test.astype('float32') / 255.0
```

```
x_train = x_train.reshape(-1, 28, 28, 1)
```

```
x_test = x_test.reshape(-1, 28, 28, 1)
```

28, 28 comes from width, height, 1 comes from the number of channels

-1 means that the length in that dimension is inferred.

This is done based on the constraint that the number of elements in an ndarray or Tensor when reshaped must remain the same.

```

# each image is a row vector (784 elements) and there are lots of such rows (let it be n, so there are 784n
elements). So TensorFlow can infer that -1 is n.
# converting the training_images array to 4 dimensional array with sizes 60000, 28, 28, 1 for 0th to 3rd
dimension.
x_train.shape
(60000, 28, 28)
x_test.shape
(10000, 28, 28, 1)
y_train.shape
(60000,)
y_test.shape
(10000,)
# We will use a convolutional neural network (CNN) to classify the fashion items.
# The CNN will consist of multiple convolutional layers followed by max pooling,
# dropout, and dense layers. Here is the code for the model:
model = keras.Sequential([
    keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    # 32 filters (default), randomly initialized
    # 3*3 is Size of Filter
    # 28,28,1 size of Input Image
    # No zero-padding: every output 2 pixels less in every dimension
    # in Paramter shwon 320 is value of weights: (3x3 filter weights + 32 bias) * 32 filters
    # 32*3*3=288(Total)+32(bias)= 320
    keras.layers.MaxPooling2D((2,2)),
    # It shown 13 * 13 size image with 32 channel or filter or depth.
    keras.layers.Dropout(0.25),
    # Reduce Overfitting of Training sample drop out 25% Neuron
    keras.layers.Conv2D(64, (3,3), activation='relu'),
    # Deeper layers use 64 filters
    # 3*3 is Size of Filter
    # Observe how the input image on 28x28x1 is transformed to a 3x3x64 feature map
    # 13(Size)-3(Filter Size )+1(bias)=11 Size for Width and Height with 64 Depth or filter or channel
    # in Paramter shwon 18496 is value of weights: (3x3 filter weights + 64 bias) * 64 filters
    # 64*3*3=576+1=577*32 + 32(bias)=18496
    keras.layers.MaxPooling2D((2,2)),
    # It shown 5 * 5 size image with 64 channel or filter or depth.
    keras.layers.Dropout(0.25),

```



```

keras.layers.Conv2D(128, (3,3), activation='relu'),
# Deeper layers use 128 filters
# 3*3 is Size of Filter
# Observe how the input image on 28x28x1 is transformed to a 3x3x128 feature map
# It show 5(Size)-3(Filter Size )+1(bias)=3 Size for Width and Height with 64 Depth or filter or
channel
# 128*3*3=1152+1=1153*64 + 64(bias)= 73856
# To classify the images, we still need a Dense and Softmax layer.
# We need to flatten the 3x3x128 feature map to a vector of size 1152
keras.layers.Flatten(),
keras.layers.Dense(128, activation='relu'),
# 128 Size of Node in Dense Layer
# 1152*128 = 147584
keras.layers.Dropout(0.25),
keras.layers.Dense(10, activation='softmax')
# 10 Size of Node another Dense Layer
# 128*10+10 bias= 1290
])
model.summary()
Model: "sequential"

```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
dropout (Dropout)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
dropout_1 (Dropout)	(None, 5, 5, 64)	0

conv2d_2 (Conv2D)	(None, 3, 3, 128)	73856
flatten (Flatten)	(None, 1152)	0
dense (Dense)	(None, 128)	147584
dropout_2 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1290

Total params: 241,546

Trainable params: 241,546

Non-trainable params: 0

Compile and Train the Model

After defining the model, we will compile it and train it on the training data.

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))

1875 is a number of batches. By default batches contain 32 samples. $60000 / 32 = 1875$

Finally, we will evaluate the performance of the model on the test data.

test_loss, test_acc = model.evaluate(x_test, y_test)

print('Test accuracy:', test_acc)

313/313 [=====] - 3s 10ms/step - loss: 0.2606 - accuracy: 0.9031

Test accuracy: 0.9031000137329102

Conclusion- In this way we can classify fashion clothing into categories using CNN.

Assignment 4:

Recurrent neural network (RNN) Use the Google stock prices dataset and design a time series analysis and prediction system using RNN.

Objective

The objective of this assignment is to design and implement a Recurrent Neural Network (RNN) for time series forecasting using historical Google (GOOG) stock prices. The model should learn from past stock data to predict future closing prices accurately.

Theory

Recurrent Neural Networks (RNNs) are designed for sequential data where previous inputs influence future outputs. Unlike traditional feedforward neural networks, RNNs maintain a hidden state that gets updated with each input step, giving them a form of memory. This makes them ideal for time series tasks like stock prediction. In this project, a simple RNN is trained on 60-day windows of Google's historical closing prices to predict the price on the 61st day. The model is trained using normalized data and evaluated using Mean Squared Error (MSE).

Workflow

1. Data Collection: Google stock data is retrieved using the yfinance library (2018–2024).
2. Preprocessing:
 - Use only the 'Close' column.
 - Normalize values with MinMaxScaler.
 - Create input sequences (60 previous days → 1 target day).
3. Model Design:
 - Use a SimpleRNN layer with 50 units and a Dense output layer.
4. Training:
 - Use 80% of data for training, 20% for testing.
 - Train the model using Adam optimizer and MSE loss.
5. Prediction & Visualization:
 - Predict on test data.
 - Plot predicted prices vs real prices.

Source Code:

```
import yfinance as yf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

# Download Google stock data (GOOG) for the last 5 years
print("Downloading stock data...")
data = yf.download('GOOG', start='2018-01-01', end='2024-01-01')
data_close = data[['Close']]

# Normalize the data
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data_close)

# Create sequences
def create_sequences(data, seq_length):
    x, y = [], []
    for i in range(seq_length, len(data)):
        x.append(data[i-seq_length:i, 0])
        y.append(data[i, 0])
    return np.array(x), np.array(y)

sequence_length = 60
X, y = create_sequences(scaled_data, sequence_length)

# Reshape for RNN input
X = np.reshape(X, (X.shape[0], X.shape[1], 1))

# Train-Test Split
train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

# Build the RNN model
model = Sequential([
    SimpleRNN(units=50, return_sequences=False, input_shape=(X_train.shape[1], 1)),
    Dense(1)
])

model.compile(optimizer='adam', loss='mean_squared_error')
model.summary()

# Train the model
print("Training the model...")
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_test, y_test))

# Predict
```

```
print("Predicting stock prices...")
predicted_stock_price = model.predict(X_test)
predicted_stock_price = scaler.inverse_transform(predicted_stock_price.reshape(-1, 1))
real_stock_price = scaler.inverse_transform(y_test.reshape(-1, 1))

# Plotting results
plt.figure(figsize=(12,6))
plt.plot(real_stock_price, color='red', label='Real Google Stock Price')
plt.plot(predicted_stock_price, color='blue', label='Predicted Google Stock Price')
plt.title('Google Stock Price Prediction using RNN')
plt.xlabel('Time')
plt.ylabel('Google Stock Price')
plt.legend()
plt.tight_layout()
plt.show()
```

Output:

Results

- The model successfully learned patterns from the past data.
- The predicted stock prices closely followed the actual prices with minor deviations.
- The line graph showed a strong alignment between real and predicted prices, demonstrating the model's ability to generalize patterns over time.

Conclusion

This assignment demonstrates the effectiveness of Recurrent Neural Networks in time series forecasting, especially for financial data. Although simple RNNs may not capture long-term dependencies as well as LSTMs or GRUs, they still offer valuable insights for short-term forecasting. Further improvements could include using more advanced RNN variants, tuning hyperparameters, or incorporating additional features like volume or technical indicators.